

Game Strategy Optimiser Using Dynamic Programming

DONE BY

G.SESHA SAI TEJA	AP24110011657
SHAIK MUSHTAQ	AP24110011007
NAGAMANI SANKAR	AP24110011622
KARTHIK POTNURU	AP24110010869
K.DURGA MANIKANTA	AP24110010878

B.Tech CSE 2nd Year 4th semester

SECTION – "U"

SUBMITTED TO

RAKESH RAMA RAJU

CODING SKILLS – 2 (CSE 206)

Prepared in the partial fulfilment

of the Project Based Learning of Course CSE 206 – Coding skill-2



SRM UNIVERSITY, AP

Introduction

In recent times, optimization is considered a vital factor in the effective solution of practical problems. Indeed, various optimization algorithms and methodologies are frequently used to make intelligent decisions with respect to specific conditions. Optimization algorithms include Dynamic Programming and various methods based on the concept of Knapsack.

The current project, titled as *Game Strategy Optimizer*, tries to provide a simulation of the problem that has to be solved by the player in a game. In this game, the battlefield consists of two dimensional array, which is associated with rewards or penalties depending on cells' values. In this case, it is necessary to find a profitable area in this matrix and choose appropriate resources from this area.

To do so, the *Maximum Sum Submatrix* algorithm can be applied within this project, since this algorithm works according to the principles of Dynamic Programming.

Overall, the current project shows that several approaches to optimization can be combined in order to effectively solve a particular problem.

ABSTRACT

The current project offers an approach that utilizes simulations in order to create a tool, named *Game Strategy Optimizer*, dedicated to making effective decisions through Dynamic Programming methods. In the model used in the system, the battlefield is represented as a grid with cells carrying numerical values that stand for benefits or risks.

In the first part of the project, the Maximum Sum Submatrix algorithm is used to find the most profitable area on the battlefield based on calculating the maximum sum for a submatrix. The obtained area can be further used by the game character to complete its mission successfully.

In the second part of the project, the items available on the battlefield (such as weapons and other resources) are found according to the previously selected area of interest. To select the best items, the 0/1 Knapsack problem algorithm is implemented, which maximizes the value of all selected items while remaining within capacity.

Objectives

1. Create a system based on games and implementing decision-making algorithms.
2. Create a representation of a battlefield using a two-dimensional matrix with reward and risk values.
3. Implement the Maximum Sum Submatrix algorithm to find the most profitable submatrix.
4. Analyze the data in the form of a grid to be able to use it in the simulation of uncertain worlds with changing parameters.
5. Get the available resources (items) dynamically from the selected submatrix.
6. Implement the 0/1 Knapsack problem to get an optimal set of items depending on the capacity.
7. Achieve maximum usage of resources through maximum optimization.
8. Combine both approaches in one system to optimize in multiple stages.
9. Explore various approaches for decision-making problems and their implementation using dynamic programming algorithms.
10. Develop a program that allows getting data either manually or automatically.

Modules Of The System

1. Input Module

This module takes input from the user regarding the battle field grid. The user can decide whether to use manual input or random number generator.

2. Grid Generation Module

Generates battlefield grid using random numbers to represent rewards and risks in different regions.

3. Grid Display Module

Helps in displaying the generated grid.

4. Maximum Sum Submatrix Module (DP)

Uses dynamic programming for finding the region in the grid having maximum sum.

5. Region Analysis Module

Analyses the chosen region and determines what kinds of items can be found there.

6. Item Extraction Module

Translates the information regarding items into actual items having weight and value.

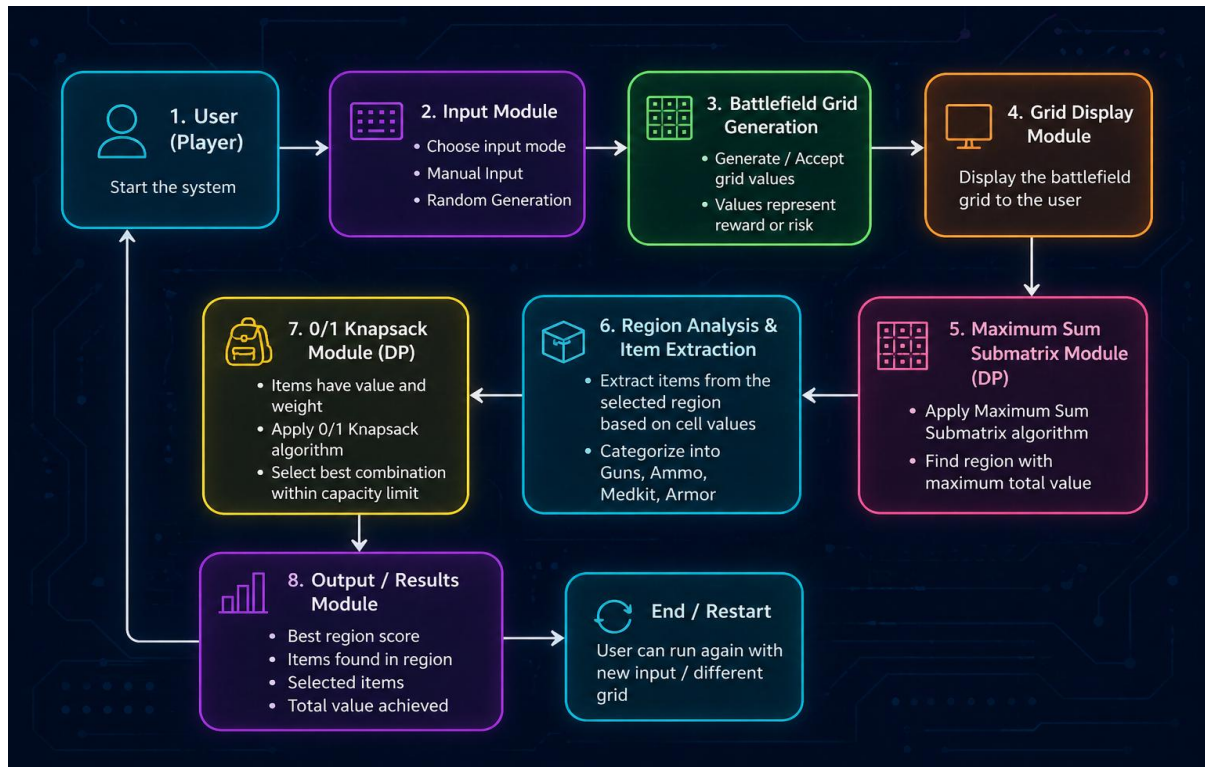
7. Knapsack Module (0/1 DP)

Applies 0/1 knapsack algorithm to choose optimal items in the given capacity.

8. Output Module

Prints the output containing region score, items in the region, chosen items, and their total value.

Data flow diagram



Explantation (Highlights in Level 1):

1. Input Module (Interaction with the User)

Takes inputs from the user for size of the grid for the battlefield.

Offers an option to the user whether to enter the values or to generate it randomly.

2. Battlefield Grid Creation

Creates/receives values for grid rewards (positive) and risks (negative).

Creates the scenario for further analysis.

3. Displaying Grid Values

Displays the grid that was created/generated.

Helps the user get visual of the battlefield.

4. Maximum Sum Submatrix Module (Dynamic Programming)

Operates the grid in order to detect the area with maximum sum.

Finds the best area of advantage for the player.

5. Analysis of Region Values and Item Extraction

Evaluates the values from the best area.

Extracts items like Guns, Ammo, Medkit, and Armor within the specific ranges.

6. Item List Generation Module

Creates an inventory list of items by converting the values with appropriate weights and values.

7. 0/1 Knapsack Module (Dynamic Programming)

Finds the optimal selection of items according to weight/capacity constraints.

8. Output Module

Displays results containing:

- Maximum Region Score
- Items extracted from the region
- Selection of Items

Implementation

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <limits.h>

#define MAX 50

// ----- PRINT GRID -----

void printGrid(int matrix[MAX][MAX], int rows, int cols) {

    printf("\nBattlefield Grid:\n");

    for(int i = 0; i < rows; i++) {

        for(int j = 0; j < cols; j++) {

            printf("%4d ", matrix[i][j]);

        }

        printf("\n");

    }

}

// ----- RANDOM GRID -----

void generateGrid(int matrix[MAX][MAX], int rows, int cols) {

    for(int i = 0; i < rows; i++) {

        for(int j = 0; j < cols; j++) {

            matrix[i][j] = (rand() % 31) - 10;

        }

    }

}
```

```
    }  
}  
  
// ----- MANUAL INPUT -----  
  
void inputGrid(int matrix[MAX][MAX], int rows, int cols) {  
    printf("Enter grid values:\n");  
    for(int i = 0; i < rows; i++) {  
        for(int j = 0; j < cols; j++) {  
            scanf("%d", &matrix[i][j]);  
        }  
    }  
}  
  
// ----- KADANE -----  
  
int kadane(int arr[], int n, int *start, int *finish) {  
    int sum = 0, maxSum = INT_MIN;  
    int local_start = 0;  
    *finish = -1;  
    for(int i = 0; i < n; i++) {  
        sum += arr[i];  
        if(sum < arr[i]) {  
            sum = arr[i];  
            local_start = i;  
        }  
    }
```

```
    if(sum > maxSum) {  
        maxSum = sum;  
        *start = local_start;  
        *finish = i;  
    }  
}  
  
if(*finish != -1)  
    return maxSum;  
  
maxSum = arr[0];  
*start = *finish = 0;  
for(int i = 1; i < n; i++) {  
    if(arr[i] > maxSum) {  
        maxSum = arr[i];  
        *start = *finish = i;  
    }  
}  
  
return maxSum;  
}  
  
// ----- MAX SUM SUBMATRIX -----  
  
int maxSumSubmatrix(int matrix[MAX][MAX], int rows, int cols,  
                    int *top, int *bottom, int *left, int *right) {
```

```
int maxSum = INT_MIN;
for(int l = 0; l < cols; l++) {
    int temp[MAX] = {0};
    for(int r = l; r < cols; r++) {
        for(int i = 0; i < rows; i++)
            temp[i] += matrix[i][r];
        int start, finish;
        int sum = kadane(temp, rows, &start, &finish);
        if(sum > maxSum) {
            maxSum = sum;
            *left = l;
            *right = r;
            *top = start;
            *bottom = finish;
        }
    }
}

return maxSum;
}

// ----- ITEM STRUCT -----
struct Item {
    char name[20];
```

```
int value;

int weight;

};

// ----- 0/1 KNAPSACK -----

int knapsack01(struct Item items[], int n, int capacity) {

    int dp[n+1][capacity+1];

    for(int i = 0; i <= n; i++) {

        for(int w = 0; w <= capacity; w++) {

            if(i == 0 || w == 0)

                dp[i][w] = 0;

            else if(items[i-1].weight <= w) {

                int include = items[i-1].value + dp[i-1][w - items[i-1].weight];

                int exclude = dp[i-1][w];

                dp[i][w] = (include > exclude) ? include : exclude;

            } else {

                dp[i][w] = dp[i-1][w];

            }

        }

    }

    printf("\nSelected Items:\n");

    int w = capacity;
```

```
for(int i = n; i > 0 && w > 0; i--) {
    if(dp[i][w] != dp[i-1][w]) {
        printf("%s (taken)\n", items[i-1].name);
        w -= items[i-1].weight;
    }
}
return dp[n][capacity];
}
// ----- MAIN -----
int main() {
    int matrix[MAX][MAX];
    int rows, cols, choice;
    srand(time(0));
    printf("Enter battlefield grid size (rows cols): ");
    scanf("%d %d", &rows, &cols);
    printf("\n1. Manual Input\n2. Random Generation\nEnter choice: ");
    scanf("%d", &choice);
    if(choice == 1)
        inputGrid(matrix, rows, cols);
    else if(choice == 2)
        generateGrid(matrix, rows, cols);
    else {
```

```
printf("Invalid choice\n");  
return 0;  
}  
printGrid(matrix, rows, cols);  
  
// ----- FIND BEST REGION -----  
int top, bottom, left, right;  
int bestSum = maxSumSubmatrix(matrix, rows, cols,  
                               &top, &bottom, &left, &right);  
printf("\nBest battlefield region score = %d\n", bestSum);  
  
// ----- EXTRACT ITEMS FROM REGION -----  
int guns=0, ammo=0, medkit=0, armor=0;  
  
for(int i = top; i <= bottom; i++) {  
    for(int j = left; j <= right; j++) {  
        int val = matrix[i][j];  
        if(val >= 15) guns++;  
        else if(val >= 10) armor++;  
        else if(val >= 5) medkit++;  
        else if(val > 0) ammo++;  
    }  
}
```

```
}  
  
printf("\nItems found in region:\n");  
  
printf("Guns=%d Ammo=%d Medkit=%d Armor=%d\n",  
       guns, ammo, medkit, armor);  
  
  
// ----- CREATE ITEMS -----  
  
struct Item items[4];  
  
int n = 0;  
  
if(guns > 0)  
    items[n++] = (struct Item){ "Guns", 60 * guns, 40 * guns};  
  
if(ammo > 0)  
    items[n++] = (struct Item){ "Ammo", 40 * ammo, 20 * ammo};  
  
if(medkit > 0)  
    items[n++] = (struct Item){ "Medkit", 50 * medkit, 30 * medkit};  
  
if(armor > 0)  
    items[n++] = (struct Item){ "Armor", 50 * armor, 40 * armor};  
  
int capacity = 120;  
  
printf("\nPlayer Capacity = %d\n", capacity);  
  
int maxValue = knapsack01(items, n, capacity);  
  
printf("\nMaximum loot value = %d\n", maxValue);  
  
return 0;  
  
}
```


Output

```
Enter battlefield grid size (rows cols): 3 3
```

```
1. Manual Input
2. Random Generation
Enter choice: 2
```

```
Battlefield Grid:
```

```
13   8   -7
-1   -1  -2
1    3   -2
```

```
Best battlefield region score = 23
```

```
Items found in region:
```

```
Guns=0 Ammo=2 Medkit=1 Armor=1
```

```
Player Capacity = 120
```

```
Selected Items:
```

```
Armor (taken)
```

```
Medkit (taken)
```

```
Ammo (taken)
```

```
Maximum loot value = 180
```

```
...Program finished with exit code 0
```

```
Press ENTER to exit console.
```

Conclusion

This project, titled Game Strategy Optimizer, is a successful example demonstrating the application of Dynamic Programming algorithms in solving complex decision-making problems. As a result of merging two algorithms – Maximum Sum Submatrix and 0/1 Knapsack – the program successfully conducts multistage optimization. First of all, the best area of the battlefield for the player is determined and then the optimal selection of items from among those in the selected region is performed.

An important aspect demonstrated by this project is the necessity of performing analysis before making a decision, since the selection of the area has a direct impact on the choice of items available for the player. Moreover, this project demonstrates a rational way to deal with limited capacity.

In general, the project provides insight into how to apply algorithmic concepts in real life scenarios, such as game strategy creation. This project also serves as a good example of the applicability of Dynamic Programming to solving complex problems related to spatial analysis and resource allocation.

GITHUB LINK:

<https://github.com/seshasai2006/Coding-Skills-2>